Derived or Observed: A Duty Written From First Principles Covers the Hazards It Can Derive and Misses the Ones Only Observation Reveals
A Reproducible Single-Turn Assay of Procedural-Specification Format and Hazard Coverage

Sophie D. Neilson
Independent Researcher
ORCID: 0009-0001-2430-1743

September 2026

Abstract

A procedure can be written in more than one form. We compare two forms of a *duty* (a gated procedure that governs a class of work) for repairing a confirmed software defect. The *annotated* form is written by reading a hazard registry and citing each entry it guards against, so the finished procedure depends on that registry to be understood. The *cartographer* form is written from first-principles analysis of the task, cites nothing, and is self-sufficient: an executor needs no external reference to follow it. We ask which hazards each form's procedure actually prevents. A local open-weight model (Qwen3.8-27B) writes the procedure in a single completion over a bare inference endpoint, and a blind judge scores, for each of ten pre-registered hazards, whether the procedure contains a step that would prevent it. The hazards are split in advance into seven that are derivable from the task and three that are not: a routing obligation upstream of the task, a session-close recording obligation, and a durability distinction learnable only from accumulated observation. The annotated form covers every hazard (210 of 210 derivable, 90 of 90 non-derivable). The cartographer form covers the derivable hazards at 134 of 210, no better than a model given no method at all (127 of 210, p=0.55), and the non-derivable hazards at 1 of 90. The gap between the forms is 0.36 on derivable hazards and 0.99 on non-derivable ones (Fisher p=2×10^{-51}). The cartographer form's distinctive property is self-sufficiency, not coverage: it cites no registry entry in any run, where the annotated form cites all ten in every run. We then test a fix the form's proposers specified but never ran: a meta-taboo scan that names two hazard classes, routing and session-close. It recovers exactly those two (routing 0 to 27 of 30; session-close 1 to 25 of 30) and not the third, the pure observation-only durability hazard it does not name (0 to 0 of 30). Naming a class of hazard lets first-principles derivation reach it; a hazard whose badness is only apparent from accumulated observation, and that belongs to no named class, stays out of reach. The finding is a direction-and-rate result on one defect specimen and one model; the materials, runs, and scoring are public and reproduce on a 24 GB GPU with no paid API.

1 Introduction

Two people can write the same procedure in different forms. One writes by consulting a registry of known hazards and citing each one the procedure guards against; the reader of that procedure must know the registry to understand it. The other writes from first principles, analyzing the task and encoding each step so it stands on its own; the reader needs nothing but the procedure. The second form is more portable, since it carries its own justification, and a research program that writes machine-followed procedures (*duties*) prefers it for that reason. This paper asks what the portable form costs. A procedure written without the registry can only guard against the hazards its author can derive. If some hazards are not derivable from the task, if their danger is apparent only from having seen them happen, the self-sufficient form has no way to reach them.

We make this concrete with a repair-duty: a duty governing how an office repairs a confirmed software defect at a known locus. We hand a local open-weight model one of two writing methods and a repair task, and it emits the repair-duty in a single completion. We then score, for each of ten pre-registered hazards, whether the produced duty contains a gate that would prevent it. The ten hazards are pre-split into those derivable from the repair act and those that are not. The question is whether the form of the writing method decides which hazards the produced duty covers, and whether that decision falls along the derivable/not-derivable line.

This reformulates an earlier two-run study in the same program, which reported the pattern from a single pair of hand-written procedures and proposed but never ran a fix. We restate it as a quantified, pre-registered, reproducible assay and run the fix.

1.1 Related Work

Whether the structure of a written specification changes how a model follows it is an active question for coding agents. Gloaguen et al. (2026) evaluate repository-level context files and find that an imperative instruction is followed while descriptive repository overviews add little. McMillan (2026) runs a factorial study of file-structure variables in coding-agent configuration files. Geng et al. (2026) measure the pragmatic influence of instruction wording, and Pecher et al. (2026) show classification behavior shifts with prompt underspecification. Our contribution is narrower and orthogonal to these: we hold the task fixed and vary only whether the procedure-writing method reads a hazard registry, and we measure hazard coverage of the produced procedure rather than task accuracy. We searched Google Scholar and arXiv (September 2026) for work on whether the form of a procedural specification determines which hazards it covers, as distinct from whether it is followed; we found none directly on point, and the coding-agent configuration studies above are the nearest neighbors. The duty, taboo, and cartographer/annotated vocabulary is internal to this research program; the reformulated study it draws on is Neilson (in preparation).

2 Materials and Method

2.1 The reformulation

The earlier study used a multi-turn agent that read files and wrote a procedure to disk. That subject cannot run on the reproducible endpoint this program requires, so the assay is recast as a single-turn generation: the model receives a writing method and the repair task, and emits the complete repair-duty as its one reply. There is no harness, no tool use, and no second turn; the produced text is the artifact scored. This preserves the question (does the writing method's form decide hazard coverage?) while fitting an endpoint anyone can reproduce.

2.2 Conditions and materials

The independent variable is the writing method. Information is not matched across conditions, by design: the two forms are defined by their relationship to the hazard registry, so the annotated method carries the registry and the cartographer method does not. This asymmetry is the object of study, not a confound. A hazard whose danger is only apparent from accumulated observation cannot be derived from the task; withholding the registry from the cartographer condition is what makes "does first-principles derivation reach this hazard?" a real question rather than "will the model copy a list it can see?" A pilot with the registry placed in the shared prompt confirmed the latter: the model copied the observation-only hazards and the effect vanished.

- **baseline**: the repair task and the defect only, no writing method and no registry. The floor.
- **annotated**: the annotated method, carrying the ten-hazard registry: read it, open with a hazard table, write one gate per hazard, and cite the registry entry in each gate body.
- **cartographer**: the cartographer method, no registry: derive the failure modes from the repair act, encode each as a self-sufficient gate, cite nothing.
- **cartographer+scan**: the cartographer method plus a meta-taboo scan that names two hazard classes, routing (upstream of the act) and session-close obligations (after the act), and directs a structural gate for each.

The shared task names one defect: `calculate_average` divides by `len(values)` and raises on an empty list, locus confirmed. The writing methods and the registry are reproduced verbatim in Appendix A.

2.3 The ten hazards and the pre-registered split

Each hazard names a divergence point a repair can cross. Seven are derivable from analyzing the repair act (a confirmed locus before editing; a declared pass criterion; a scope boundary; fixing the canonical source not a call site; a revised root cause before a second attempt; recorded completion evidence; verification by observed result not claim). Three are not: a *routing* obligation upstream of the act (confirm a repair, not another task, is what is needed); a *session-close* obligation (record a constraint discovered during the repair before closing); and a *durability* distinction (advisory versus structural fixes), whose badness is learnable only from accumulated observation. The split, the divergence points, and which hazards the scan names are fixed before the run in `studies/cartographer-duty-format/taboo_set.json`.

2.4 Model and Sampling

The subject is Qwen3.8-27B (Q4_K_M GGUF) served over a bare `llama.cpp` raw completion endpoint (no chat template, no tools) with thinking pinned off in the published wrapper. The complete sampler chain is declared in the study definition and sent with every request: temperature 0.8; `min_p` 0.05 as the sole live truncation stage; `top_k`, `top_p`, `typical_p`, `top_n_sigma`, all penalties, and the XTC and DRY stages off; one seed per replicate (1 through 30); a 4096-token cap. Each condition ran 30 replicates. The server ran on a GPU (RTX 3090); recorded throughput held between 37.9 and 40.8 tokens per second across all 120 runs, with no step down to a CPU rate, and no response was truncated at the cap (the longest was 3806 predicted tokens). The served build, read from the endpoint's `props` at the session, was `b9445-af6528e6d`; the per-row build field was empty in this run's record, a recorded gap noted here rather than inferred. The probe image digest and the substrate probe are in the run record. The repository was on commit `56eb443` with a dirty working tree when the run launched, disclosed here; the committed materials and instrument are at the follow-up commit named in Data Availability.

2.5 Scoring

Two measurements. The first is deterministic: a script counts how many of the ten registry slugs each produced duty cites (`slug_c2.py`), which needs no judge. The second is coverage: for each duty and each hazard, does the duty contain a gate whose action would prevent that hazard's divergence point? Naming a hazard is not coverage; the gate's action is. Coverage was judged by Claude (Opus 4.8), blind to condition: the 120 duties were shuffled and stripped of their condition labels into an opaque-id set before judging, and unblinded only for analysis. An independent second rater, also blind and given a clean rubric with no worked example, re-scored a stratified 40-duty sample; agreement is reported in Section 3.

2.6 Agents and Models

Role	Model	Configuration	Date
Subject	Qwen3.8-27B	Q4_K_M GGUF, `llama.cpp` build `b9445-af6528e6d` (per `props`), raw completion, thinking off, GPU (RTX 3090), context 32768.	2026-09-11
Slug scorer (C2)	Deterministic rule	`slug_c2.py`; exact registry-slug match on the response text; no judgment.	2026-09-11
Coverage judge	Claude Opus 4.8	Blind to condition (shuffled, de-labelled set); four batches over the 120 duties. Not a treatment arm.	2026-09-12
Second rater	Claude Opus 4.8	Independent session, blind, clean rubric with no worked example; stratified 40-duty sample. Same family as the primary judge (see Limitations).	2026-09-12
Author and operator	Claude Opus 4.8	Authored the materials, ran the study, wrote the coverage rubric. Not a treatment subject.	2026-09-12

Table 1: Agents and models. The subject is the only treatment arm and is a local open-weight model; Claude is used only to judge and to operate, never as a treatment condition.

3 Results

3.1 Slug citation is a clean, deterministic separation

The annotated method cites all ten registry slugs in every one of its 30 runs (mean 10.0 slugs). The baseline, cartographer, and cartographer+scan conditions cite none, in any run. The self-sufficiency the cartographer form is written for holds exactly: its produced duties carry no dependency on the registry, where the annotated form's carry all of it.

3.2 Coverage splits along the derivable line

Table 2 gives coverage by hazard class. The annotated method covers everything. The cartographer method covers the derivable hazards at 134 of 210 and the non-derivable at 1 of 90. The annotated-minus-cartographer gap is 0.36 on derivable hazards and 0.99 on non-derivable ones; on the pooled non-derivable hazards the difference is near-total (Fisher exact, $p=2\times10^{-51}$). The cartographer method's derivable-hazard coverage (134 of 210) is not distinguishable from the no-method baseline (127 of 210, Fisher $p=0.55$): writing from first principles does not raise coverage over giving the model no method at all. What it changes is the dependency, not the coverage.

3.3 The scan recovers the classes it names

The meta-taboo scan names two hazard classes. Table 3 shows it recovers exactly those two and not the third. Routing coverage rises from 0 of 30 to 27 of 30, and session-close from 1 of

Condition	**Derivable (7 hazards)**	**Non-derivable (3 hazards)**
baseline	127/210	4/90
annotated	210/210	90/90
cartographer	134/210	1/90
cartographer+scan	133/210	52/90

Table 2: Hazard coverage by class, as covered/judged, over 30 runs per condition and 7 or 3 hazards. Derivable hazards are those a first-principles analysis of the repair act can reach; non-derivable are the routing, session-close, and durability hazards.

30 to 25 of 30. The durability hazard, which the scan does not name and which is learnable only from observation, stays at 0 of 30. Naming a class of hazard lets first-principles derivation reach it; the hazard that belongs to no named class remains unreachable.

Hazard	**Class**	**Scan names it**	**cartographer**	**+scan**	**Fisher p**
routing	meta/routing	yes	0/30	27/30	9.2×10^{-14}
session-close	session-close	yes	1/30	25/30	1.2×10^{-10}
durability	observation-only	no	0/30	0/30	1.0

Table 3: Cartographer versus cartographer+scan on the three non-derivable hazards. The scan recovers the two classes it names and not the one it does not.

3.4 Two nominally derivable hazards behave like the hard ones

The derivable/not-derivable split predicts the broad pattern, but it is not perfectly clean. Two hazards labelled derivable are missed by the cartographer method almost as often as the non-derivable ones: fixing the canonical source rather than a call site (7 of 30) and revising the root cause before a second attempt (2 of 30). Only the annotated method, carrying the registry, covers them reliably (30 of 30 each). These two sit closer to the hard end than their label implies: they are derivable in principle but subtle enough that a from-scratch analysis usually does not surface them. We report the split as pre-registered and flag these two rather than re-labelling them after the fact.

3.5 Inter-rater agreement

The independent second rater, blind and given a clean rubric with no worked example, re-scored a stratified 40-duty sample (10 per condition). Agreement with the primary judge was 372 of 400 hazard calls (0.93; Cohen's $\kappa = 0.85$). The disagreements concentrate on one borderline hazard, whether recorded completion evidence is required (29 of 40); the hazards that carry the finding agree strongly, including the observation-only durability hazard at 40 of 40 and routing at 39 of 40. Because the second rater saw no worked example, the agreement indicates the primary coverage calls are not an artifact of the worked example in the primary rubric.

4 Discussion

The result draws a line under a portable procedure. A duty written from first principles, needing no external reference, is exactly as good as its author's ability to derive hazards from the task, and no better, on this evidence, than a model given no method at all. Its value is that it carries no dependency, not that it prevents more. The hazards it misses are the ones a task cannot teach: an upstream routing check, a session-close obligation, and a durability distinction that only accumulated observation reveals. The annotated form reaches all of them because it copies them from a registry that already remembers them.

The scan result sharpens this. A hazard that a first-principles author misses is not always unreachable; it is unreachable until it is named. Naming the class ("check for routing hazards," "check for session-close obligations") gives the derivation a place to look, and coverage of those classes jumps from near zero to most runs. What naming does not rescue is the hazard whose content, not just its existence, must be observed: the advisory-versus-structural durability distinction stays at zero even with the scan running, because knowing to look for durability hazards does not tell you what makes an advisory fix fragile. That is the residue that a self-sufficient format cannot, in principle, carry.

4.1 Limitations

One defect specimen, one model, single-turn generation. This is a direction-and-rate finding on the `calculate_average` empty-list defect as written for Qwen3.8-27B; the derivable/not-derivable boundary may fall differently for another domain, another model, or a multi-turn agent that can consult a registry by choice rather than having it withheld. The information asymmetry between the forms is deliberate and is discussed in the Method; a consequence is that the non-derivable coverage gap is partly expected, since a hazard that must be observed cannot be derived, so the informative quantities are how much of the derivable set the cartographer form recovers (most) and whether naming a class recovers the rest (for two of three, yes).

The coverage judge and the second rater are both Claude (Opus 4.8), a weaker form of independence than a separate architecture would give: the two readings agree, but they are not cross-architecture. We rested the mechanism on the deterministic slug count (Section 3), which needs no judge, and leave a local cross-architecture coverage rater to future work. The primary coverage judge saw a worked example object in its rubric that could have anchored its calls; the second rater was given no such example precisely to test that, and the two agree (Section 3). The pre-registered derivable split is imperfect for two hazards, disclosed above and not re-labelled.

5 Conclusion

A self-sufficient procedure covers the hazards its author can derive from the task and misses the ones that can only be observed. Writing from first principles does not, on this evidence, prevent more failures than no method at all; it removes the dependency on a registry, and with it the reach to any hazard the registry alone remembers. Naming a class of hazard restores the reach for hazards whose existence is the only thing that must be supplied; it does not restore reach for a hazard whose content must be observed. A portable format and complete hazard coverage are, in this setting, two different things.

5.1 Data Availability

The complete study, comprising the writing methods, the shared task and defect, the pre-registered hazard set and split, the design and pre-registration documents, the run records with per-row provenance, the produced duties, the blind judging set and map, the deterministic slug scorer, the coverage grid, and the analysis script, is available at `https://github.com/sophdn/corpos-lab` under the `studies/cartographer-duty-format/` directory. The instrument change that added the three format conditions is in the same repository under `internal/`.

References

Geng, Y., Abend, O., Hovy, E., and Frermann, L. (2026). Measuring Pragmatic Influence in Large Language Model Instructions. *arXiv preprint arXiv:2602.21223*.

Gloaguen, T., Mündler, N., Müller, M., Raychev, V., and Vechev, M. (2026). Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents? *arXiv preprint arXiv:2602.11988*.

McMillan, D. (2026). Instruction Adherence in Coding Agent Configuration Files: A Factorial Study of Four File-Structure Variables. *arXiv preprint arXiv:2605.10039*.

Neilson, S. D. (in preparation). Canon Suppression in Corpus-Loaded Assessment: A Two-Assessor Study.

Pecher, B., Spiegel, M., Belanec, R., and Cegin, J. (2026). Revisiting Prompt Sensitivity in Large Language Models for Text Classification: The Role of Prompt Underspecification. *arXiv preprint arXiv:2602.04297*.

A Materials

The writing methods (annotated, cartographer, cartographer+scan), the shared scenario with the defect, and the ten-hazard registry are reproduced verbatim in the study directory named in Data Availability (`studies/cartographer-duty-format/materials/`). The pre-registered hazard split and the scan-target flags are in the same directory's `taboo_set.json`, and the coverage judging brief is in `scores/JUDGE_BRIEF.md`.